

Casting Values Between Data Types in Java

In Java, **casting** refers to converting a value of one data type to another. This can be done in two ways:

1. **Automatic Type Promotion (Implicit Casting)**
 2. **Manual Type Casting (Explicit Casting)**
-

1. Automatic Type Promotion (Implicit Casting)

Automatic Type Promotion happens when a smaller data type is assigned to a larger data type. Java performs this type of casting automatically because it's safe, meaning there's no loss of data.

- **Syntax:** Java automatically converts smaller types to larger types. For example, an `int` can be automatically promoted to a `long`, and a `float` can be promoted to a `double`.
- **Example:**

```
public class Main {  
    public static void main(String[] args) {  
        int x = 10;  
        double y = x; // Automatic promotion from int to double  
  
        System.out.println("The value of y: " + y);  
    }  
}
```

- **Explanation:**
 - The `int` variable `x` is automatically promoted to a `double` when assigned to `y`.
 - **No data loss occurs** since a `double` can store all values of an `int` without losing precision.
 - **Automatic Promotions:**
 - **byte** → **short** → **int** → **long** → **float** → **double** (Wider types automatically accept values of narrower types)
-

2. Manual Type Casting (Explicit Casting)

Manual Type Casting (also known as **Explicit Casting**) is used when a larger data type is assigned to a smaller one. This type of casting **requires explicit intervention** from the programmer because there could be data loss during the conversion.

- **Syntax:** You perform an explicit cast by placing the target data type in parentheses before the value or variable.

```
targetType variable = (targetType) value;
```

- **Example:**

```
public class Main {  
    public static void main(String[] args) {  
        double a = 9.99; // Larger type  
        int b = (int) a; // Manual casting from double to int, data  
        loss occurs (decimal part is discarded)  
  
        System.out.println("The value of b: " + b);  
    }  
}
```

- **Explanation:**
 - The value of `a` (which is a `double`) is explicitly cast to an `int`. During this casting, the decimal part `.99` is truncated, and only the integer part `9` is stored in `b`.
- **Manual Type Casting Requires Caution:**
 - **Potential data loss:** Converting from a `double` to an `int` (or other narrowing conversions) may lose the fractional part.
 - **Narrowing Types:** It's used when a larger data type is assigned to a smaller data type, such as from `double` to `int`.

Type Casting Examples

Example 1: Automatic Promotion

```
public class Main {  
    public static void main(String[] args) {  
        int num = 100;  
        double result = num; // Implicit casting (int to double)  
  
        System.out.println("The double value is: " + result); // 100.0  
    }  
}
```

```
}  
}
```

- **Explanation:** The `int` value is automatically promoted to a `double`.

Example 2: Explicit Casting

```
public class Main {  
    public static void main(String[] args) {  
        double num = 100.99;  
        int result = (int) num; // Explicit casting (double to int)  
  
        System.out.println("The int value is: " + result); // 100 (decimal  
part is truncated)  
    }  
}
```

- **Explanation:** The `double` value `100.99` is cast to an `int`, and the fractional part `.99` is discarded.

Example 3: Casting a Larger Type to a Smaller Type

```
public class Main {  
    public static void main(String[] args) {  
        long largeNumber = 100000L;  
        int smallerNumber = (int) largeNumber; // Manual casting (long to  
int)  
  
        System.out.println("The int value is: " + smallerNumber);  
    }  
}
```

- **Explanation:** When casting from `long` (which is 64 bits) to `int` (32 bits), there is a risk of **data loss** if the value exceeds the range of an `int` (-2^{31} to $2^{31}-1$).

Type Promotion and Casting Rules

1. Widening (Automatic Promotion):

- Narrow types are automatically promoted to wider types:
 - `byte` → `short` → `int` → `long` → `float` → `double`
 - Example: `int x = 10; double y = x;`

2. Narrowing (Manual Casting):

- Wider types need to be manually cast to narrower types:

- double → float → long → int → short → byte
- Example: double x = 10.5; int y = (int) x;

3. Char to int:

- char can be automatically promoted to int since a char is just a small integer value (representing Unicode values).

```
char c = 'A';  
int i = c; // Implicit casting from char to int  
System.out.println("The integer value of 'A' is: " + i); // 65  
(Unicode value of 'A')
```

4. Casting Between Numeric Types:

- When casting between numeric types (byte , short , int , long , float , double), larger types can be assigned to smaller types, but **you must use explicit casting** for narrowing.
- Example:

```
float f = 10.75f;  
int i = (int) f; // Explicit cast  
System.out.println(i); // 10
```